# 5장 서버

분산형 버전 관리 깃은 다양한 유형의 저장소를 지원합니다. 저장소는 크게 로컬 저장소와 서버 저장소로 구분할 수 있습니다. 이 장에서는 서버 저장소를 알아보겠습니다.

5.1 서버 저장소

5.2 깃허브 서버 준비

5.3 깃허브 연동 및 원격 등록

5.4 서버 전송

5.5 자동으로 내려받기

5.6 수동으로 내려받기

5.7 순서

5.8 정리

## 5.1 서버 저장소

서버 저장소는 다른 말로 원격(remote) 저장소라고도 합니다. 서버 저장소는 로컬 저장소의 코드를 복제한 복사본이라고 할 수 있습니다. 서버를 이용하면 코드를 안전하게 보관할 수 있습니다. 또 서버에 있는 소스 코드는 다른 사람들과 공유하고 협업하여 개발을 진행할 수도 있습니다.

### 5.1.1 협업 저장소

고객이 요구하는 소프트웨어 품질이 높아지면서 최근 프로젝트 개발 규모가 점점 커지고 있습니다. 규모가 큰 프로젝트는 혼자서 모두 개발하기 어렵고, 시간과 노력이 많이 필요합니다. 여러 명이 같이 협업하여 개발한다면 적은 시간으로 좀 더 좋은 품질의 소프트웨어를 만들 수 있습니다. 깃은 여러 개발자와 협업하려고 탄생한 도구입니다.

과거와 달리 요즘 컴퓨터는 항상 인터넷에 접속되어 있습니다. 하지만 아직도 365일 24시간 인터넷에 연결하여 작업할 수 없는 개발 환경도 많이 있습니다. 깃은 이 두 가지 환경을 고려하여 분산형 모델을 선택했습니다.

### 5.1.2 연속된 작업

원격 저장소가 있다면 언제 어디에서든지 개발을 이어서 할 수 있습니다. 사무실에서 개발 중인 코드를 서버에 저장하고, 집에 와서는 사무실에서 작업하고 서버에 올린 코드를 자신의 컴퓨터에 동기화할 수 있습니다. 이처럼 사무실, 집, 다른 여러 컴퓨터에 코드를 동기화하고 연속된 작업을 이어 갈 수 있습니다.

▼ 그림 5-1 원격 저장소에 연결




깃은 분산된 저장소 여러 개를 하나로 통합하고, 최신 코드를 배포할 수 있습니다. 서버 저장소는 여러 컴퓨터에 동일한 깃 저장소를 복제하고, 작업한 결과물을 다시 서버로 통합합니다.

### 5.1.3 새 멤버

깃의 분산형 관리 체계는 다수의 사람과 협업하는 데 매력적입니다. 기존 프로젝트에 새로운 멤버가 참여할 때, 지금까지 작업한 소스 코드의 마지막 버전을 공유해야 합니다.

기존에는 코드를 공유하려고 이메일, 외부 저장 장치 등을 이용했지만, 이제는 깃의 원격 저장소 주소만 알려 주면 모두 해결됩니다. 원격 저장소로 모든 구성원에게 코드의 최종 결과물을 동기화합니다.

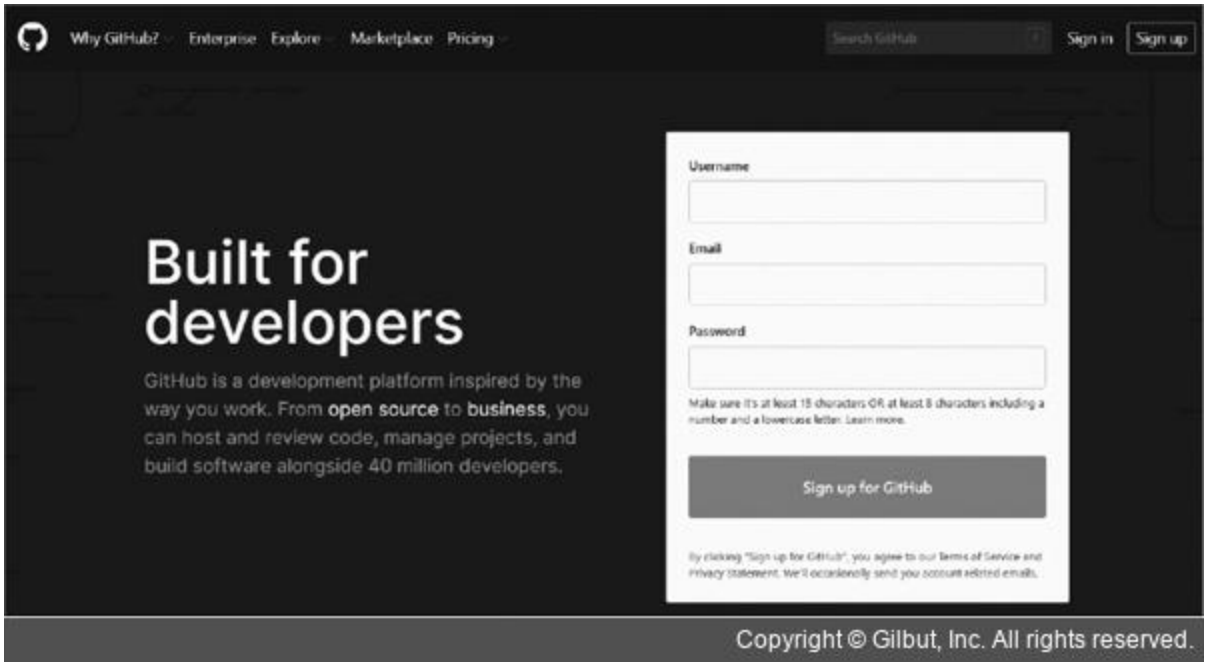
## 5.2 깃허브 서버 준비

독립적인 깃 서버를 직접 운영하여 사용할 수 있습니다. 하지만 365일 안정적인 서버를 운영하는 것은 쉽지 않습니다. 직접 서버를 운영하지 않아도 전문적인 깃 호스팅으로 서버를 대체할 수 있습니다. 호스팅을 받으면 직접 서버를 관리하지 않아도 쉽게 원격 저장소를 운영할 수 있습니다.

### 5.2.1 깃허브

'깃'은 잘 모르지만 '깃허브'는 한 번쯤 들어 보았을 것입니다. 깃허브는 대표적인 깃호스팅 서비스입니다. 깃허브의 모든 서비스는 무료로 사용할 수 있습니다. 서비스를 사용하려면 먼저 회원 가입이 필요합니다. 깃허브 공식 사이트는 https://github.com 입니다. 공식 사이트에 접속하면 다음 화면이 나옵니다.

▼ 그림 5-2 깃허브 공식 사이트




회원 가입을 하려면 이메일 주소가 필요합니다. 사용자 이름은 영문으로 작성하며, 저장소를 구별하는 주소 값으로 사용합니다. 일반적으로 개별 깃허브 주소는 다음과 같이 표현합니다.

https://github.com/사용자이름

회원 가입을 완료했다면 이메일로 본인 인증 과정을 거쳐야 합니다. 회원 가입 후 입력했던 이메일 저장소에서 새 이메일을 확인해 주세요. 이메일 목록에서 [GitHub] Please verify your email address.로 표시된 이메일을 클릭합니다. 이메일 안에 있는 Verify email address를 클릭합니다. 등록한 이메일 계정의 소유자인지 확인하는 절차입니다.

### 5.2.2 저장소 생성

깃허브에 새로운 저장소를 생성하는 방법을 알아보겠습니다. 공개용 저장소로 생성하여 실습할 것입니다. 공개된 저장소는 무제한으로 생성해서 사용할 수 있습니다. 비공개용은 제약이 조금 있고, 일부는 유료 서비스입니다.

로그인한 상태에서 새로운 저장소를 생성합시다. 대시보드에서 New 버튼을 클릭하거나 + > New repository를 선택합니다.

▼ 그림 5-3 저장소 생성 메뉴



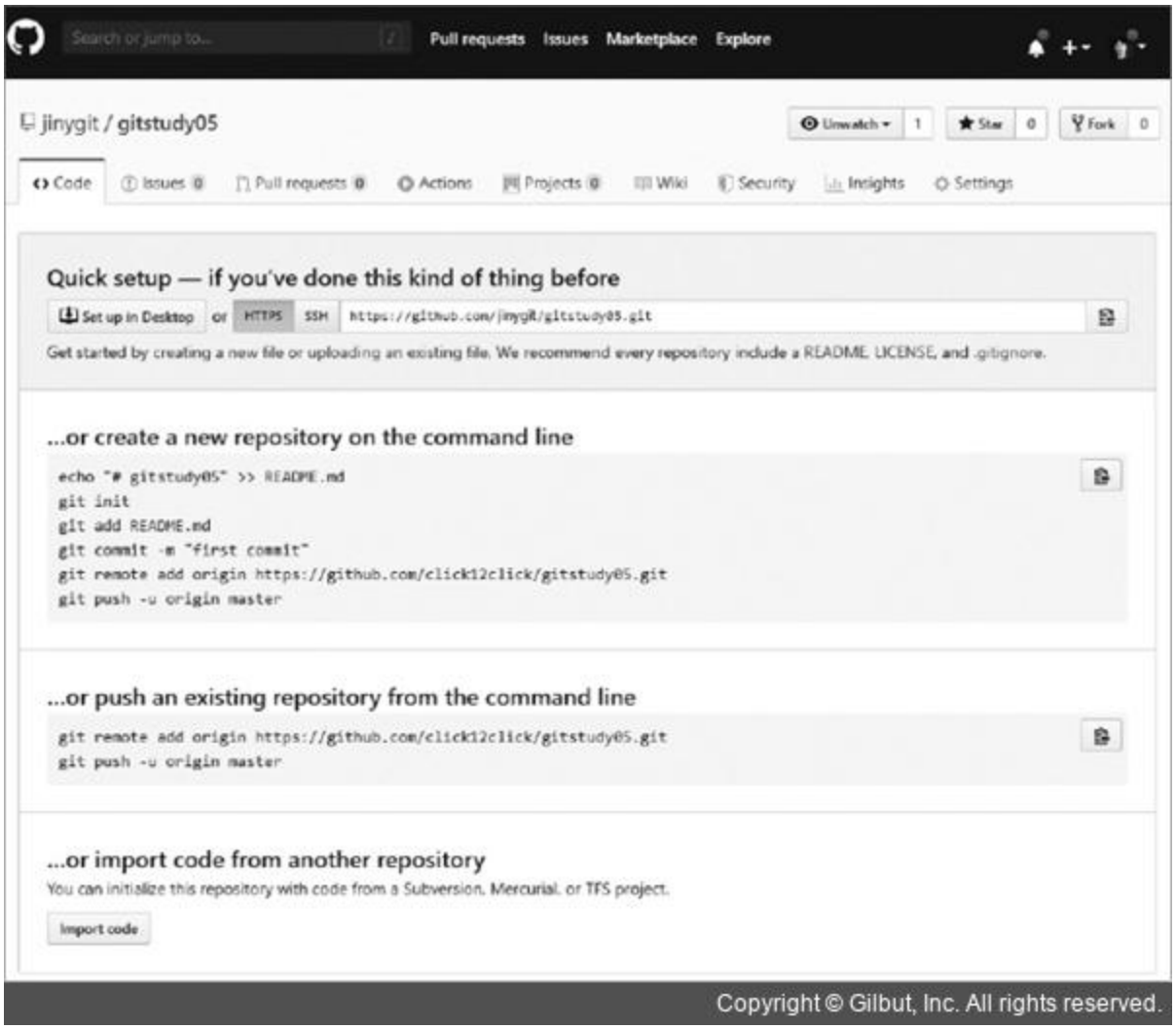
## 5.3 깃허브 연동 및 원격 등록

깃허브에 새 저장소를 생성했다면 이제 로컬 저장소와 연결해야 합니다. 기존 로컬 저장소와 연결하거나 새 로컬 저장소를 생성하여 연결할 수도 있습니다.

### 5.3.1 로컬 저장소

5.2절에서 새 저장소를 생성하면 깃허브에 다음 화면이 나옵니다. 이 화면은 저장소를 생성할 때 README 체크 여부에 따라 달라집니다. README를 체크하지 않으면 다음과 같이 초기화 및 복제 방법을 안내합니다.

▼ 그림 5-5 깃허브에서 새 저장소 생성



### 5.3.2 프로토콜

서버와 통신하려면 프로토콜을 사용해야 합니다. 깃은 서버와 통신할 수 있는 다양한 프로토콜을 지원합니다. 깃은 기본적으로 Local, HTTP, SSH, Git 네 종류의 전송 방식을 지원합니다.2

Local(로컬)

로컬 컴퓨터에 원격 저장소를 생성하는 것을 의미합니다. 이 방식은 자신의 컴퓨터를 NFS (Network File System) 등 서버로 이용할 때 편리합니다.

로컬 저장소를 서버로 이용할 때는 폴더 경로만 입력하면 됩니다.

$ git remote add 원격저장소별칭 폴더경로

로컬은 간단하게 원격 서버를 구축할 수 있을 뿐만 아니라 빠른 동작이 가능합니다. 하지만 모든 자료가 자신의 컴퓨터에 집중되는 위험도 있습니다.

HTTP

깃은 HTTP 방식의 프로토콜을 지원합니다. HTTP는 SSH처럼 많이 사용하는 프로토콜 중 하나입니다. 깃허브, 비트버킷 같은 호스팅 서비스도 기본 HTTP 프로토콜을 지원합니다.

서버에 접속하려면 로그인 절차를 거쳐야 합니다. HTTP는 기존 아이디와 비밀번호만으로 접속자를 인증하여 처리할 수 있습니다. HTTP는 익명으로도 처리할 수 있으며, 계정을 이용하여 처리할 수도 있습니다.

### 5.3.3 원격 저장소의 리모트 목록 관리

깃은 원격 저장소(서버)를 관리하는 데 remote 명령어를 사용합니다. remote 명령어를 사용하면 현재 연결된 원격 저장소 목록을 확인할 수 있습니다. 동시에 등록과 취소 등 작업을 할 수 있습니다.

remote 명령어에는 다양한 옵션이 있으며, -help 옵션으로 확인할 수 있습니다. 명령어 하나로 다수의 리모트 작업을 하기 때문에 몇 가지 옵션은 반드시 알고 넘어가야 합니다. 5.3.5~5.3.8절에 걸쳐 알아볼 것입니다.

remote 명령어를 독립적으로 사용하면 연결된 원격 저장소의 이름(별칭)을 출력합니다. 간단하게 원격 저장소 목록만 확인할 때 편리합니다.

$ git remote

-v 옵션을 같이 사용하면 원격 저장소의 별칭 이름과 URL을 확인할 수 있습니다.

$ git remote -v

깃은 복수의 원격 저장소를 연결하여 사용할 수 있습니다. 리모트 저장소가 여러 개 있을 때는 목록을 모두 출력합니다. 하지만 저장소의 권한 정보까지는 알 수 없습니다.

### 5.3.4 주소와 별칭

로컬 저장소에 원격 저장소(서버)를 등록하려면 서버 주소가 필요합니다. 깃허브 같은 저장소를 이용해 보면 프로토콜 + 도메인 주소 형태로 된 것을 알 수 있습니다. 로컬에 서버 저장소를 생성할 때는 폴더 경로를 사용할 수 있습니다.

- 별칭: 원격 서버의 주소는 긴 문자열로 되어 있습니다. 매번 원격 서버에서 작업할 때마다 긴 문자열을 입력하는 것은 피곤합니다. 간략하게 긴 서버 URL 문자열을 별칭으로 만들어 사용할 수 있습니다.
- origin: origin은 대표적으로 사용하는 별칭입니다. 기본적으로 원격 서버와 연결할 때 별칭으로 origin을 사용하는 것을 많이 볼 수 있습니다. 하지만 자신의 목적에 따라 다른 이름을 사용해도 괜찮습니다.

### 5.3.5 원격 저장소에 연결

이제 원격 저장소와 연결해 보겠습니다. 원격 저장소와 연결하려면 add 옵션을 사용합니다.

$ git remote add 원격저장소별칭 원격저장소URL

원격 저장소를 추가할 때는 인자 값으로 원격 저장소 별칭과 원격 저장소의 URL을 같이 입력합니다. 별칭은 앞에서도 이야기했듯이, 서버 주소가 길기 때문에 쉽게 작업하려고 사용하는 별명입니다.

```
infoh@hojin MINGW64 /e/gitstudy05 (main)


$ git remote add origin https://github.com/jinygit/gitstudy05.git 자신의 서버 주소를 입력




infoh@hojin MINGW64 /e/gitstudy05 (main)


$ git remote -v 원격 저장소 목록 확인


origin  https://github.com/jinygit/gitstudy05.git (fetch)


origin  https://github.com/jinygit/gitstudy05.git (push)
```

원격 저장소가 연결되면 fetch와 push 두 주소를 출력합니다. push(푸시)는 서버로 전송하는 동작을 의미하고, fetch(페치)는 반대로 서버에서 가지고 오는 동작을 의미합니다.

별칭은 중복하여 선택할 수 없습니다.

### 5.3.6 소스트리에서 원격 브랜치

원격 저장소를 등록하면 기존 main 브랜치와 달리 또 하나의 브랜치가 표시됩니다. 그림 5-6은 서버에 전송했던 실습 화면을 캡처한 것입니다.[3]

▼ 그림 5-6 원격 브랜치




main는 현재 로컬 저장소를 의미합니다. 그리고 local/main 같은 별칭/브랜치는 원격 저장소의 브랜치를 의미합니다. 즉, 로컬 저장소와 서버 저장소를 구분하여 표시합니다. 이것으로 서로 동기화한 시점을 판별할 수 있습니다.

Note

---

브랜치 개념은 나중에 다시 자세히 설명합니다. 원격 저장소를 등록하면 원격 브랜치가 자동 생성된다는 것만 알고 넘어갑시다.

### 5.3.7 별칭 이름 변경과 정보

별칭은 긴 문자열의 서버 주소를 대체합니다. 등록된 서버의 별칭 이름은 다시 변경할 수 있습니다. rename 옵션을 같이 사용합니다.

$ git remote rename 변경전 변경후

remote 명령어는 간략한 원격 저장소 정보만 출력합니다. 원격 저장소의 좀 더 상세한 정보를 확인하고 싶다면 show 옵션을 같이 사용합니다.

$ git remote show 원격저장소별칭

예를 들어 다음과 같이 remote show 명령어를 실행하면 상세한 정보를 출력합니다.[4]

예

```
infoh@hojin MINGW64 /e/gitstudy05 (main)

$ git remote show origin

* remote origin

  Fetch URL: https://github.com/jinygit/gitstudy05.git

  Push  URL: https://github.com/jinygit/gitstudy05.git

  HEAD branch: main

  Remote branch:

    main tracked

  Local ref configured for 'git push':

    main pushes to main (up to date)
```

### 5.3.8 원격 서버 삭제

로컬 저장소는 복수의 원격 저장소와 연결할 수 있습니다. 깃을 사용하다 보면 풀 리퀘스트(pull request),[5] 테스트 등 목적으로 임시 등록된 원격 저장소들도 있습니다. 등록된 원격 저장소는 rm 옵션으로 삭제할 수 있습니다.

$ git remote rm 원격저장소별칭

등록한 origin 저장소를 삭제하고, 목록을 다시 확인해 보겠습니다.

```
infoh@hojin MINGW64 /e/gitstudy05 (main)


$ git remote rm origin 원격 저장소 삭제




infoh@hojin MINGW64 /e/gitstudy05 (main)


$ git remote -v 원격 저장소 목록 확인
```

이처럼 등록, 변경, 삭제하여 다수의 원격 저장소를 관리할 수 있습니다.

다음 실습을 대비하여 삭제된 원격 저장소를 다시 등록해 놓습니다.

```
infoh@hojin MINGW64 /e/gitstudy05 (main)


$ git remote add origin https://github.com/jinygit/gitstudy05.git 재등록
```

> ⚠️ **[최신 깃허브 정책 중요 안내] 비밀번호 로그인 차단 및 PAT 인증 필수**
> 현재 GitHub은 보안상의 이유로 계정 패스워드를 통한 HTTPS 인증을 완전히 차단했습니다.
> 터미널이나 소스트리(SourceTree)에서 원격 로그인(비밀번호 입력)을 요구받는 경우, 기존 비밀번호 대신 반드시 **Personal Access Token (PAT)**을 비밀번호 란에 입력해야 합니다.
>
> 🔗 **토큰 발급 방법:**
> GitHub 우측 상단 프로필 이미지 클릭 ➔ **Settings** ➔ 왼쪽 최하단 **Developer Settings** ➔ **Personal Access Tokens** (Classic 또는 Fine-grained) ➔ **Generate new token** 실행 (필요한 repo 권한 체크 후 생성 및 토큰 값을 복사하여 비밀번호 란에 붙여넣기)
> 👉 **GitHub 토큰 관리 페이지 바로가기**

## 5.4 서버 전송

로컬 저장소의 커밋을 원격 저장소로 전송하는 방법을 알아보았습니다. 원격 저장소가 연결되면 로컬 저장소의 커밋들을 업로드할 수 있습니다.

### 5.4.1 push: 서버에 전송

푸시(push)는 원격 저장소로 커밋된 파일들을 업로드하는 동작입니다. 원격 저장소와 연결된 서버 목록을 확인해 봅시다.

```
infoh@hojin MINGW64 /e/gitstudy05 (main)


$ git remote -v


origin  https://github.com/jinygit/gitstudy05.git (fetch)

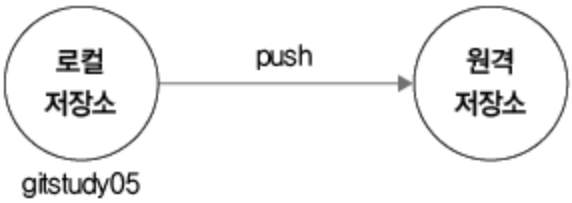

origin  https://github.com/jinygit/gitstudy05.git (push)
```

origin https://github.com/jinygit/gitstudy05.git (push)처럼 업로드가 가능한 원격 저장소가 등록된 것을 확인할 수 있습니다. 원격 저장소로 로컬 깃 저장소의 내용을 전송할 때는 push 명령어를 사용합니다.

$ git push 원격저장소별칭 브랜치이름

별칭 이름을 가지는 서버의 main 브랜치에 현재 브랜치를 업로드합니다. 브랜치(branch)는 다음 장에서 상세하게 설명합니다.

▼ 그림 5-7 푸시 동작



## 5.5 자동으로 내려받기

이번에는 반대로 원격 저장소에서 커밋된 코드를 내려받는 방법을 알아보겠습니다.

### 5.5.1 clone: 복제

복제는 기존 저장소를 이용하여 새로운 저장소를 생성하는 방법 중 하나입니다. 하지만 일반적인 복제와 원격 저장소를 복제하는 방법은 조금 차이가 있습니다. 몇 가지 추가 정보를 같이 설정해야 합니다.

복제할 때는 clone 명령어를 사용합니다. clone 명령어는 초기화 init 명령어 외에 원격 서버 접속에 필요한 추가 설정을 자동으로 수행합니다. 서버의 연결 설정을 마친 후 서버 안에 있는 모든 커밋된 코드 이력들을 한 번에 내려받습니다.

앞에서 업로드한 원격 저장소를 다른 이름의 로컬 저장소로 복제해 보겠습니다.

```
$ cd 메인폴더

$ mkdir gitstudy05_clone 복제할 새 폴더 만들기

$ cd gitstudy05_clone/



infoh@hojin MINGW64 /e/gitstudy05_clone

$ git clone https://github.com/jinygit/gitstudy05.git . 저장소 복제

Cloning into '.'...

remote: Enumerating objects: 3, done.

remote: Counting objects: 100% (3/3), done.

remote: Total 3 (delta 0), reused 3 (delta 0), pack-reused 0

Unpacking objects: 100% (3/3), done.
```

### 5.5.2 pull: 서버에서 내려받기

복제는 원격 저장소에서 모든 내용을 한 번에 내려받습니다. 복제 후 원격 저장소의 갱신된 내용을 추가로 내려받으려면 pull 명령어를 사용해야 합니다.

pull 명령어는 로컬 저장소보다 최신인 갱신된 원격 저장소의 커밋 정보를 현재 로컬 저장소로 내려받습니다. pull 명령어를 주기적으로 사용하면 최신 커밋 정보로 로컬 저장소를 유지할 수 있습니다.

$ git pull

실습을 위해 원본 로컬 저장소로 이동합니다.

```
infoh@gitstudy MINGW64 /e/gitstudy05_clone (main)


$ cd ../ gitstudy05 원본 폴더로 이동




infoh@gitstudy MINGW64 /e/gitstudy05 (main)


$ code server.htm VS Code 실행
```

server.htm

```
<h1>서버 실습입니다.</h1>


<h2>오늘도 좋은 하루입니다.</h2>
```

## 5.6 수동으로 내려받기

원격 저장소 내용을 내려받는 방법은 크게 두 가지입니다. pull(풀)과 fetch(페치)입니다. 이 두 방법의 차이는 병합을 자동 처리하는지 여부입니다. 병합은 8장에서 자세히 설명합니다. 이 장에서는 페치 병합 방법만 알고 넘어갑니다.

### 5.6.1 자동 병합

풀은 원격 서버에서 현재 커밋보다 더 최신 커밋 정보가 있을 때 내려받습니다. 내려받은 커밋 정보는 임시 영역에 저장합니다. 스테이지 영역이 아닌 원격 저장소를 위한 전용 임시 브랜치가 따로 있습니다.

내려받은 최신 커밋들을 현재 브랜치로 자동으로 병합 처리합니다. 여기서 병합은 원격 서버 파일과 로컬 파일을 하나로 합치는 과정입니다. 혼자서 개발하는 프로젝트는 pull 명령어만으로도 편리하게 사용할 수 있습니다.

하지만 여러 개발자와 협업으로 코드를 작성할 때 pull 명령어를 사용한 자동 병합은 가끔씩 문제가 생깁니다. 여러 개발자와 협업하는 과정에서 pull 명령어가 자동으로 브랜치 병합을 하지 못하고 충돌이 발생하기도 합니다. 충돌은 8장 병합 부분에서 자세히 설명합니다.

이처럼 pull 명령어로 자동 병합을 하지 못할 때는 페치 방식을 사용해야 합니다.

### 5.6.2 fetch: 가져오기

fetch(페치)는 원격 저장소에서 코드를 수동으로 내려받는 작업을 합니다. 페치는 원격 저장소에서 커밋된 코드를 임시 브랜치로 내려받습니다. 내려받은 후 현재 브랜치와 자동 병합하지 않습니다.

$ git fetch 원격저장소URL

그럼 페치 동작을 실습해 봅시다. 먼저 실습 원본 저장소로 이동합니다.

```
infoh@hojin MINGW64 /e/gitstudy05_clone (main)


$ cd ../ gitstudy05 원본 폴더로 이동




infoh@hojin MINGW64 /e/gitstudy05 (main)


$ code server.htm VS Code 실행
```

server.htm

```
<h1>서버 실습입니다.</h1>


<h2>오늘도 좋은 하루입니다.</h2>


<h2>가끔은 하늘을 보세요.</h2>
```

수정된 파일을 스테이지 영역에 등록과 동시에 커밋합니다.

```
infoh@hojin MINGW64 /e/gitstudy05 (main)

$ git commit -am "look sky"

[main 668b5ef] look sky

 1 file changed, 1 insertion(+)
```

### 5.6.3 merge 명령어로 수동 병합

페치는 데이터를 내려받기만 할 뿐 자동 병합하지 않습니다. 내려받은 커밋을 로컬 저장소에 적용하려면 병합 명령을 실행해야 하는데, merge 명령어를 사용합니다. 병합은 8장에서 자세히 설명하므로 여기서는 간단히 알아보겠습니다.

merge 명령어는 다음과 같이 사용합니다.

$ git merge 원격저장소별칭/브랜치이름

fetch 명령어로 내려받은 커밋을 로컬 저장소에 병합하겠습니다. 다음과 같이 merge 명령어를 입력합니다.

```
infoh@hojin MINGW64 /e/gitstudy05_clone (main)


$ git merge origin/main 임시 원격 브랜치 병합


Updating 6a947b8..668b5ef


Fast-forward


 server.htm | 1 +


 1 file changed, 1 insertion(+)
```

## 5.7 순서

원격 저장소에는 다수의 개발자가 동시에 커밋을 푸시할 수 없습니다. 여러 명이 협력해서 개발할 때는 순차적으로 푸시해야 합니다.

### 5.7.1 최신 상태

먼저 원격 저장소에 푸시하려면 자신의 로컬 저장소를 최신 상태로 유지해야 합니다. 즉, 자신의 저장소가 원격 저장소의 커밋보다 최신이어야 한다는 의미입니다. 최신이라는 의미를 좀 더 구체적으로 알아봅시다.

누군가 내 저장소보다 먼저 커밋하여 새로운 커밋으로 서버를 갱신했습니다. 하지만 나는 간발의 차이로 갱신된 서버 정보를 가지고 있지 않습니다. 푸시는 서버의 마지막 커밋과 푸시되는 커밋을 병합합니다. 이때 내 커밋은 서버의 최신 커밋보다 늦은 커밋으로 밀리게 됩니다. 커밋이 순차적이지 않을 때 깃은 푸시 동작을 거부합니다.

푸시 동작이 거부되면 풀 또는 페치 작업으로 자신의 로컬 저장소를 갱신해 주어야 합니다. 갱신 후에 다시 푸시합니다.

예를 들어 보겠습니다. 개발자 1, 개발자 2가 함께 코드를 개발합니다. 개발자 1이 코드를 수정했고, 개발자 2도 동시에 코드를 수정했습니다. 그리고 개발자 1이 코드를 푸시하여 원격 저장소를 갱신했습니다. 이후 개발자 2가 자신의 코드를 푸시하면 오류가 발생합니다. 개발자 1이 원격 저장소를 먼저 갱신하여 커밋 + 1 상태가 되었기 때문입니다. 따라서 개발자 2의 로컬 컴퓨터는 개발자 1이 갱신한 상태로 자신의 저장소를 pull 명령어로 업데이트한 후 푸시할 수 있습니다.

### 5.7.2 충돌 방지

깃이 최신 상태에서만 푸시를 허용하는 것은 충돌을 방지하기 위해서입니다. 원격 저장소의 커밋을 내려받는 풀 작업은 내려받은 커밋들을 현재 브랜치로 자동 병합합니다. 이때 커밋 내용이 순차적이지 않으면 병합 과정에서 충돌이 발생합니다.

개발자 1과 개발자 2가 서로 다른 작업을 했다면 충돌이 없을 수도 있지만, 같은 영역을 동시에 수정했다면 개발자 2가 풀 작업을 할 때 무조건 충돌이 발생합니다. 개발자 2는 충돌을 해결한 후 커밋하고 푸시해야 하는데, 이를 해결하기가 어렵습니다. 따라서 깃은 이를 더 쉽게 해결할 수 있도록 푸시할 때 커밋의 순차적 기록을 확인합니다.

push 명령어를 사용할 때는 충돌을 최소화하기 위해 미리 원격 저장소를 확인합니다. 새로운 커밋 갱신이 없는지 pull 명령어를 사용하여 지속적으로 확인하는 것이 좋습니다.

최대한 충돌을 피할 수 있는 방법은 자신의 저장소와 원격 저장소의 상태를 자주 최신으로 유지하는 것입니다.

▼ 그림 5-10 권장 순서




풀과 푸시를 자주 하여 충돌을 최소한으로 줄여 나가면서 작업을 유지합시다.

소스트리를 사용하면 푸시하기 전에 자신의 저장소와 어떤 차이점이 있는지 쉽게 확인할 수 있습니다. 소스트리의 push




위에 조그마한 숫자가 표시됩니다. 숫자는 현재 로컬 저장소와 원격 저장소 간 커밋 차이점을 출력합니다.

## 5.8 정리

깃은 코드 이력을 관리해 줄 뿐만 아니라 다른 개발자와 협업 도구로도 많이 사용합니다. 다른 개발자와 협업하려면 공유 매개체의 역할을 수행할 서버가 필요합니다.

깃은 다양한 종류의 서버를 지원합니다. 깃 서버를 직접 만들 수도 있고, 인기 있는 깃 호스팅 서비스를 이용할 수도 있습니다.

깃은 서버 역할을 수행하는 원격 저장소와 커밋 정보들을 주고받습니다. 로컬 컴퓨터는 원격 저장소에 커밋 코드를 전송하거나 추가된 커밋들을 내려받을 수 있습니다. 이러한 원격 저장소 기능은 좀 더 많은 사람이 깃을 사용하게 하는 촉매제가 되었습니다. 원격 저장소를 불특정 다수를 대상으로 공유할 수도 있습니다. 오픈 소스는 깃과 공개된 원격 저장소를 사용하여 활발하게 수많은 사람과 협업할 수 있는 장점들을 제공합니다. 그래서 깃은 오픈 소스를 활성화하는 데 가장 많은 기여를 하는 협업 툴이 되었습니다.

1 깃허브는 개인 저장소 외에 조직 그룹도 생성할 수 있습니다. 따라서 저장소 소유자로 조직 그룹을 선택할 수도 있습니다.

2 깃 호스팅을 사용하더라도 HTTP와 SSH 방식은 알아 두면 좋습니다.

3 우리는 원격 저장소와 연결만 했기 때문에 소스트리에서 확인하면 결과가 조금 다릅니다.

4 우리는 원격 저장소와 연결만 했기 때문에 더 간단한 메시지를 출력합니다.

5 여러 사용자가 검토해서 여러 갈래(브랜치)의 코드를 병합하는 기능으로, 협업해서 개발할 때 사용합니다. 이러한 용어가 있다는 것만 알고 넘어가세요.